

Rekolt Tracker Application

rekolt-tracker/	
├── README.md	How to build and run from a clean clone
├── .gitignore	
├── pom.xml	or build.gradle
├── docs/	
│ ├── setup/	JDK, IDE, and git screenshots (objective 1)
│ └── 1)	
│ ├── git/	history graph (objective 2)
│ ├── collections-rationale.md	(objective 3)
│ └── design/	
│ ├── design-v1.pdf	the paper design (objective 4)
│ ├── design-v2.pdf	the design as built (objective 5)
│ └── change-log.md	deviations from design-v1
├── output/	
│ ├── run-log.txt	one timestamped line per generation
│ └── season-report.docx	one section per member, plus totals
├── src/main/java/mu/rekolt/	
│ ├── app/	entry point and console interaction
│ ├── model/	Produce, CerealProduce, PerishableProduce, CashCropProduce, Member, Delivery, Grade
│ └── service/	grading, payment, reporting, document
└── output	
└── util/	validation and formatting

- ☒ OBJECTIVE 1: Basic payment calculation
- ↓
- ☒ OBJECTIVE 2: Input + validation + menu + loops + methods
- ↓
- ☒ OBJECTIVE 3: Collections + searching + sorting
- ↓
- ☐ OBJECTIVE 4: UML + design documentation
- ↓
- ☐ OBJECTIVE 5: Abstract classes + inheritance + interfaces
- ↓
- ☐ OBJECTIVE 6: Microsoft Word report + logging

OBJECTIVE 4: UML + design documentation

1. Scope Statement

REKOLT Produce Tracker is a console-based Java application designed to help a cooperative record, process, and report agricultural produce deliveries made by its members during a 20-week season.

The system records delivery information, including the member identifier, member name, produce type, mass, quality score, and delivery week. It validates user input, determines the quality grade, applies the appropriate produce-category multiplier, and calculates the member's net payable amount.

The application supports the management of a season's deliveries using Java collections. It can calculate payment totals per member, group deliveries by member, identify distinct members, search for deliveries, sort delivery records, and remove rejected deliveries.

The system is intended for use by a cooperative clerk through a console interface. From Objective 2 onwards, the application remains running through a menu until the user chooses to exit. Invalid user input must be rejected and re-prompted rather than causing the program to terminate.

The application does not include database storage, authentication, online services, or external file-based delivery storage within the scope of the current project. The season data is held in the application's Java collections.

In short

The system allows a clerk to record information about each delivery, such as the member, type of produce, amount delivered, quality score, and delivery week. The system then uses this information to:

- Determine the quality grade of a delivery.
- Calculate the amount payable to the member.
- Keep track of all deliveries during the season.
- Calculate the total payment for each member.
- Group deliveries by member.
- Search for a particular delivery.
- Sort deliveries in different ways.
- Identify distinct members.
- Handle rejected deliveries.
- Produce season figures and reports.

2. Assumptions

The following assumptions were made when designing the system:

The system manages one season at a time.

The season contains 20 weeks.

The system supports four types of produce:

- Maize (MZE)
- Beans (BNS)
- Potatoes (POT)
- Green tea leaf (TEA)

Each member has a unique member identifier.

Each delivery has a unique delivery identifier.

A delivery with a quality score below 50 is classified as REJECT.

Rejected deliveries are still stored and included in delivery statistics, but they receive a value of zero and have no deductions.

Money values are displayed to two decimal places.

The application is operated through the console.

Invalid input should not crash the application. The user should be asked to enter the information again.

Functional Requirements

Functional requirements describe what the system should be able to do.

1. Record a Delivery

The system must allow the clerk to enter a new delivery.

The following information is required:

- Member ID
- Member name
- Produce code
- Mass in kilograms
- Quality score
- Delivery week

2. Validate Input

The system must check that the information entered by the user is valid.

If the information is incorrect, the system should display an error message and ask the user to enter it again.

3. Determine the Grade

The system must determine the grade of a delivery using its quality score.

<u>Quality score</u>	<u>Grade</u>
85 - 100	A
70 - 84	B
50 - 69	C
0 - 49	Reject

4. Calculate Payment

The system must calculate the amount payable to a member based on:

- Mass delivered
- Base price of the produce
- Grade multiplier
- Category multiplier
- Commission
- Transport levy

5. Handle Rejected Deliveries

The system must allow rejected deliveries to be recorded.

A rejected delivery:

- Has a value of zero.
- Does not receive deductions.
- Remains recorded in the system.

6. Store Deliveries

The system must store delivery records so they can be accessed and processed later during the season.

7. Calculate Member Payments

The system must calculate the total amount payable to each member based on their deliveries.

8. Group Deliveries

The system must allow deliveries to be grouped according to the member who made them.

9. Identify Members

The system must be able to identify the different members who have made deliveries.

10. Search Deliveries

The system must allow the user to search for a delivery using its delivery ID. If the delivery does not exist, the system should display an appropriate message.

11. Sort Deliveries

The system must allow delivery records to be sorted using different criteria, including their natural ordering and a custom ordering.

12. Remove Rejected Deliveries

The system must support the safe removal of rejected deliveries from the collection when required.

13. Display Season Figures

The system must display useful information about the deliveries and payments recorded during the season.

14. Exit the Application

The system must continue running until the user selects the Exit option.

Non-Functional Requirements

1. Easy to Use

The application should provide simple menus, clear instructions, and understandable error messages.

2. Reliable

The application should not crash when the user enters invalid information.

3. Accurate

All payment calculations must follow the rules given in the project specification.

4. Easy to Maintain

The program should be divided into appropriate classes and methods so that the code is easier to understand, update, and maintain.

5. Good Performance

The system should process the season's delivery information efficiently without unnecessary operations.

6. Compatible with Java

The application should run using the required Java development environment.

<u>Important Nouns</u>	<u>Important Verbs</u>
• Member	• Record
• Delivery	• Validate
• Produce	• Calculate
• Grade	• Determine
• Season	• Store
• Payment	• Search
• Quality score	• Sort
• Mass	• Group
• Week	• Remove
• Delivery ID	• Display
• Member ID	• Report
• Commission	
• Transport levy	

Classes kept

1. Member

The Member class represents a cooperative member.

It can contain information such as:

- Member ID
- Member name

2. Delivery

The Delivery class represents one delivery made by a member.

It can contain information such as:

- Delivery ID
- Member
- Produce
- Mass
- Quality score
- Week
- Grade
- Net payment

3. Produce

The produce class represents the type of agricultural produce being delivered.

The system can use specialized produce categories such as

- Cereal produce
- Perishable produce
- Cash crop produce

4. Grade

The grade class represents the quality grade assigned to a delivery.

The possible grades are:

- A
- B
- C
- REJECT

5. SeasonService

The SeasonService class manages the delivery information for the season.

It is responsible for operations such as:

- Adding deliveries
- Finding deliveries
- Grouping deliveries
- Calculating member payments

- Sorting deliveries
- Managing the season's collections

Classes Rejected

1. Payment

Payment is calculated from information already contained in a delivery, such as mass, produce, grade, and the applicable multipliers. Creating a separate Payment class would add unnecessary complexity for this project.

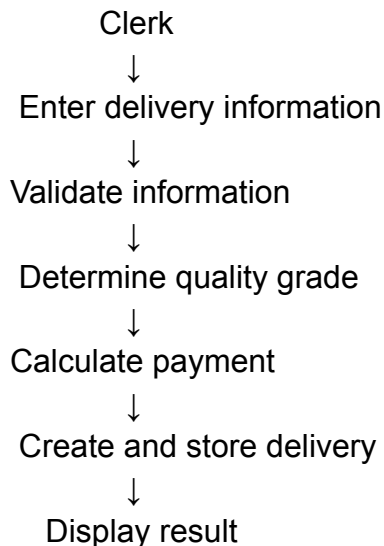
2. Commission

Commission is simply a calculation based on the delivery value. The project specifies a 5% commission, so there is no need to create a separate object just for commission.

3. TransportLevy

The transport levy is calculated using the mass of the delivery at a fixed rate of 2 MUR per kilogram. It does not need its own class because it has no independent information or behavior.

Behaviour Design



The clerk starts the process by entering the delivery information. Main receives and validates the information. The payment and grading logic is then processed using the appropriate service operations. A Delivery object is created and passed to SeasonService, which stores it in the season's delivery collection. Finally, Main displays the result to the clerk

Net Payment Pseudocode

The following pseudocode describes how the system calculates the final amount payable for a delivery.

START

INPUT massKg

INPUT pricePerKg

INPUT grade

INPUT gradeMultiplier

INPUT categoryMultiplier

$\text{baseValue} = \text{massKg} \times \text{pricePerKg}$

IF grade = "REJECT" THEN

 netPayable = 0

 commission = 0

 transportLevy = 0

ELSE

$\text{gradeValue} = \text{baseValue} \times \text{gradeMultiplier}$

$\text{categoryValue} = \text{gradeValue} \times \text{categoryMultiplier}$

$\text{commission} = \text{categoryValue} \times 0.05$

$\text{transportLevy} = \text{massKg} \times 2.00$

$\text{netPayable} = \text{categoryValue} - \text{commission} - \text{transportLevy}$

END IF

DISPLAY netPayable

END

Explanation of the Payment Calculation

The payment calculation follows a fixed order.

1. Calculate the base value

Base Value = Mass × Price per kg

2. Apply the grade multiplier

Grade Value = Base Value × Grade Multiplier

3. Apply the category multiplier

Category Value = Grade Value × Category Multiplier

4. Calculate commission

For accepted deliveries:

Commission = Category Value × 5%

5. Calculate transport levy

Transport Levy = Mass × 2 MUR

6. Calculate the final payment

Net Payable = Category Value – Commission – Transport Levy

REJECT Case

If the grade is REJECT:

Net Payable = 0

Commission = 0

Transport Levy = 0

The rejected delivery is still recorded by the system

Data Dictionary

<u>Data</u>	<u>Type</u>	<u>Unit</u>	<u>Range / Allowed Values</u>	<u>Validation</u>
Member ID	String	N/A	M-0000 format	Must follow the required member ID format
Member Name	String	N/A	Non-empty	Cannot be blank
Delivery ID	String	N/A	Unique ID	Must identify a delivery uniquely
Produce Code	String	N/A	MZE, BNS, POT, TEA	Must be one of the supported codes
Mass	double	kg	Greater than 0 and up to 5000	Must be a valid positive number
Quality Score	int	points	0–100	Must be a whole number within range
Week	int	week	1–20	Must be a whole number within range
Grade	String / Grade	N/A	A, B, C, REJECT	Determined from quality score
Price per kg	double	MUR/kg	Depends on produce	Obtained from the selected produce
Grade Multiplier	double	multiplier	Depends on grade	Selected according to grade
Category Multiplier	double	multiplier	Depends on produce category	Selected according to produce type
Commission	double	MUR	0 or 5% of category value	Not applied to rejected deliveries
Transport Levy	double	MUR	0 or mass $\times$ 2	Not applied to rejected deliveries
Net Payable	double	MUR	0 or calculated amount	Calculated by the payment rules

Produce Prices

The system uses the following base prices:

<u>Produce</u>	<u>Code</u>	<u>Price per kg</u>
Maize	MZE	30 MUR
Beans	BNS	90 MUR
Potatoes	POT	45 MUR
Green tea leaf	TEA	25 MUR

Grade Multipliers

The grade multipliers are:

<u>Grade</u>	<u>Quality Score</u>	<u>Multiplier</u>
A	85–100	1.15
B	70–84	1.00
C	50–69	0.85
REJECT	0–49	0.00

Category Multipliers

The produce category affects the value after the grade multiplier has been applied.

<u>Category</u>	<u>Produce</u>	<u>Multiplier</u>
Cereal	Maize, Beans	1.00
Perishable	Potatoes	0.90
Cash Crop	Green tea leaf	1.10

Design Rationale

Three important design decisions were made for the system.

1. Use a Delivery class

A separate delivery class was chosen because a delivery contains several pieces of information that belong together, such as the member, produce, mass, quality score, week, grade, and payment. This makes each delivery easy to store, search, and process as one object.

2. Use SeasonService

A SeasonService class was chosen to keep season-level operations separate from the user interface, keeping Main from becoming responsible for every part of the system. It can manage tasks such as:

- Storing deliveries
- Searching deliveries
- Grouping deliveries
- Calculating member payments
- Sorting deliveries

3. Use Java Collections

Java collections were chosen because the number of deliveries can change during the season, making operations such as searching, grouping, sorting, and removing records easier to implement. For example:

- List can store all deliveries.
- Map can associate members with payments or deliveries.
- Set can store unique member IDs.

UML

